Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №2

з дисципліни

«Об’єктно-орієнтоване програмування»

на тему

Розробка графічного редактора об’єктів.

Виконав:

студент групи ІМ-55

Матвіїв Ростислав Степанович

Номер в списку - 15

Перевірив:

Рекечинський Д. О.

Київ 2026

Завдання:

1. Створити у середовищі розробки новий проект з ім’ям Lab2.
2. Скомпілювати проект і отримати виконуваний файл програми.
3. Перевірити роботу програми. За потреби налагодити програму.

4. Проаналізувати та прокоментувати результати та вихідний текст програми. 5. Оформити звіт

Завдання згідно варіанту:

Номер у списку журналу: Ж = 15.

- **Масив вказівників для об’єктів `Shape`:** динамічний масив `Shape **pcshape` (15 mod 3 = 0) обсягом N = 15 + 100 = 115 об’єктів.
- **«Гумовий» слід:** пунктирна лінія чорного кольору (15 mod 4 = 3).
- **Прямокутник:**

  ввід від центру до одного з кутів (15 mod 2 = 1);

  чорний контур з білим заповненням (15 mod 5 = 0).

- **Еліпс:**

  ввід по двом протилежним кутам охоплюючого прямокутника (15 mod 2 = 1);

  чорний контур еліпсу без заповнення (15 mod 5 = 0).

- **Позначка поточного типу об’єкту:** в заголовку вікна (15 mod 2 = 1).

Робота:

Lab2.cpp

```
#include <windows.h>
#include "resource.h"
#include "shape_editor.h"

namespace {

const wchar_t kWindowClassName[] = L"Lab2MainWindowClass";
const wchar_t kWindowTitle[]     = L"OOP_lab2 (Ж=15, IM-55)";
```

```
ShapeObjectsEditor g_editor;

LRESULT CALLBACK MainWndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM
lParam) {
    switch (message) {
    case WM_LBUTTONDOWN:
        g_editor.OnLBdown(hWnd);
        return 0;

    case WM_LBUTTONUP:
        g_editor.OnLBup(hWnd);
        return 0;

    case WM_MOUSEMOVE:
        g_editor.OnMouseMove(hWnd);
        return 0;

    case WM_PAINT:
        g_editor.OnPaint(hWnd);
        return 0;

    case WM_COMMAND:
        switch (LOWORD(wParam)) {
        case ID_OBJ_POINT:
            g_editor.StartPointEditor(hWnd);
            return 0;
        case ID_OBJ_LINE:
            g_editor.StartLineEditor(hWnd);
            return 0;
        case ID_OBJ_RECT:
            g_editor.StartRectEditor(hWnd);
            return 0;
        case ID_OBJ_ELLIPSE:
            g_editor.StartEllipseEditor(hWnd);
            return 0;
        case ID_HELP_ABOUT:
            MessageBoxW(hWnd,
                L"Лабораторна робота №2\nРозробка графічного редактора
об'єктів\nАвтор: Матвіїв Р. С., група ІМ-55 (Ж=15)",
                L"Про програму", MB_OK | MB_ICONINFORMATION);
            return 0;
        case ID_FILE_EXIT:
            DestroyWindow(hWnd);
            return 0;
        }
        break;

    case WM_DESTROY:
        PostQuitMessage(0);
        return 0;
```

```
    }
    return DefWindowProcW(hWnd, message, wParam, lParam);
}

ATOM RegisterMainWindowClass(HINSTANCE hInstance) {
    WNDCLASSEXW wc = {};
    wc.cbSize        = sizeof(wc);
    wc.style         = CS_HREDRAW | CS_VREDRAW;
    wc.lpfnWndProc   = MainWndProc;
    wc.hInstance     = hInstance;
    wc.hIcon         = LoadIconW(nullptr, IDI_APPLICATION);
    wc.hCursor       = LoadCursorW(nullptr, IDC_CROSS);
    wc.hbrBackground = reinterpret_cast<HBRUSH>(COLOR_WINDOW + 1);
    wc.lpszMenuName  = MAKEINTRESOURCEW(IDR_MAINMENU);
    wc.lpszClassName = kWindowClassName;
    wc.hIconSm       = LoadIconW(nullptr, IDI_APPLICATION);
    return RegisterClassExW(&wc);
}

}

int WINAPI wWinMain(HINSTANCE hInstance, HINSTANCE, PWSTR, int nCmdShow) {
    if (!RegisterMainWindowClass(hInstance)) {
        MessageBoxW(nullptr, L"Не вдалося зареєструвати клас вікна.", L"Помилка",
MB_ICONERROR);
        return 1;
    }

    HWND hWnd = CreateWindowExW(
        0, kWindowClassName, kWindowTitle, WS_OVERLAPPEDWINDOW,
        CW_USEDEFAULT, CW_USEDEFAULT, 720, 480,
        nullptr, nullptr, hInstance, nullptr);

    if (!hWnd) {
        MessageBoxW(nullptr, L"Не вдалося створити вікно.", L"Помилка",
MB_ICONERROR);
        return 1;
    }

    ShowWindow(hWnd, nCmdShow);
    UpdateWindow(hWnd);

    MSG msg;
    while (GetMessageW(&msg, nullptr, 0, 0) > 0) {
        TranslateMessage(&msg);
        DispatchMessageW(&msg);
    }
    return static_cast<int>(msg.wParam);
}
```

shape.cpp

```cpp
#include "shape.h"

void Shape::Set(long x1, long y1, long x2, long y2) {
    xs1 = x1;
    ys1 = y1;
    xs2 = x2;
    ys2 = y2;
}

void PointShape::Show(HDC hdc) {
    for (int dx = -1; dx <= 1; ++dx) {
        for (int dy = -1; dy <= 1; ++dy) {
            SetPixel(hdc, xs1 + dx, ys1 + dy, RGB(0, 0, 0));
        }
    }
}

void LineShape::Show(HDC hdc) {
    HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));
    HPEN hOldPen = static_cast<HPEN>(SelectObject(hdc, hPen));

    MoveToEx(hdc, xs1, ys1, nullptr);
    LineTo(hdc, xs2, ys2);

    SelectObject(hdc, hOldPen);
    DeleteObject(hPen);
}

void RectShape::Show(HDC hdc) {
    HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));
    HBRUSH hBrush = static_cast<HBRUSH>(GetStockObject(WHITE_BRUSH));

    HPEN hOldPen = static_cast<HPEN>(SelectObject(hdc, hPen));
    HBRUSH hOldBrush = static_cast<HBRUSH>(SelectObject(hdc, hBrush));

    long xOpposite = 2 * xs1 - xs2;
    long yOpposite = 2 * ys1 - ys2;
    Rectangle(hdc, xOpposite, yOpposite, xs2, ys2);

    SelectObject(hdc, hOldBrush);
    SelectObject(hdc, hOldPen);
    DeleteObject(hPen);
}

void EllipseShape::Show(HDC hdc) {
    HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));
    HBRUSH hBrush = static_cast<HBRUSH>(GetStockObject(NULL_BRUSH));

    HPEN hOldPen = static_cast<HPEN>(SelectObject(hdc, hPen));
```

```
    HBRUSH hOldBrush = static_cast<HBRUSH>(SelectObject(hdc, hBrush));

    Ellipse(hdc, xs1, ys1, xs2, ys2);

    SelectObject(hdc, hOldBrush);
    SelectObject(hdc, hOldPen);
    DeleteObject(hPen);
}
```

shape.h

```
#pragma once
#include <windows.h>

class Shape {
protected:
    long xs1 = 0, ys1 = 0, xs2 = 0, ys2 = 0;
public:
    virtual ~Shape() = default;
    void Set(long x1, long y1, long x2, long y2);
    virtual void Show(HDC hdc) = 0;
};

class PointShape : public Shape {
public:
    void Show(HDC hdc) override;
};

class LineShape : public Shape {
public:
    void Show(HDC hdc) override;
};

class RectShape : public Shape {
public:
    void Show(HDC hdc) override;
};

class EllipseShape : public Shape {
public:
    void Show(HDC hdc) override;
};
```

shape_editor.cpp

```
#include "shape_editor.h"

Shape** ShapeEditor::pcshape = nullptr;
int ShapeEditor::shapeCount = 0;
```

```
void ShapeEditor::InitArray() {
    if (!pcshape) {
        pcshape = new Shape*[MY_SHAPE_ARRAY_SIZE];
        for (int i = 0; i < MY_SHAPE_ARRAY_SIZE; ++i) {
            pcshape[i] = nullptr;
        }
        shapeCount = 0;
    }
}

void ShapeEditor::FreeArray() {
    if (pcshape) {
        for (int i = 0; i < shapeCount; ++i) {
            delete pcshape[i];
        }
        delete[] pcshape;
        pcshape = nullptr;
        shapeCount = 0;
    }
}

void ShapeEditor::DrawAllShapes(HDC hdc) {
    if (!pcshape) return;
    for (int i = 0; i < shapeCount; ++i) {
        if (pcshape[i]) {
            pcshape[i]->Show(hdc);
        }
    }
}

HPEN ShapeEditor::CreateRubberPen() const {
    return CreatePen(PS_DOT, 1, RGB(0, 0, 0));
}

void ShapeEditor::OnLBdown(HWND hWnd) {
    POINT pt;
    GetCursorPos(&pt);
    ScreenToClient(hWnd, &pt);
    x1 = x2 = pt.x;
    y1 = y2 = pt.y;
    isDragging = true;
    SetCapture(hWnd);
}

void ShapeEditor::OnPaint(HWND hWnd) {
    PAINTSTRUCT ps;
    HDC hdc = BeginPaint(hWnd, &ps);
    DrawAllShapes(hdc);
    EndPaint(hWnd, &ps);
}
```

```
void PointEditor::OnMouseMove(HWND) {
}

void PointEditor::OnLBup(HWND hWnd) {
    if (!isDragging) return;
    isDragging = false;
    ReleaseCapture();

    POINT pt;
    GetCursorPos(&pt);
    ScreenToClient(hWnd, &pt);
    x2 = pt.x;
    y2 = pt.y;

    if (shapeCount < MY_SHAPE_ARRAY_SIZE) {
        pcshape[shapeCount] = new PointShape();
        pcshape[shapeCount]->Set(x2, y2, x2, y2);
        ++shapeCount;
    }
    InvalidateRect(hWnd, nullptr, TRUE);
}

void LineEditor::OnMouseMove(HWND hWnd) {
    if (!isDragging) return;

    HDC hdc = GetDC(hWnd);
    SetROP2(hdc, R2_NOTXORPEN);
    HPEN hPen = CreateRubberPen();
    HPEN hOldPen = static_cast<HPEN>(SelectObject(hdc, hPen));

    MoveToEx(hdc, x1, y1, nullptr);
    LineTo(hdc, x2, y2);

    POINT pt;
    GetCursorPos(&pt);
    ScreenToClient(hWnd, &pt);
    x2 = pt.x;
    y2 = pt.y;

    MoveToEx(hdc, x1, y1, nullptr);
    LineTo(hdc, x2, y2);

    SelectObject(hdc, hOldPen);
    DeleteObject(hPen);
    ReleaseDC(hWnd, hdc);
}

void LineEditor::OnLBup(HWND hWnd) {
    if (!isDragging) return;
    isDragging = false;
```

```
    ReleaseCapture();

    POINT pt;
    GetCursorPos(&pt);
    ScreenToClient(hWnd, &pt);
    x2 = pt.x;
    y2 = pt.y;

    if (shapeCount < MY_SHAPE_ARRAY_SIZE) {
        pcshape[shapeCount] = new LineShape();
        pcshape[shapeCount]->Set(x1, y1, x2, y2);
        ++shapeCount;
    }
    InvalidateRect(hWnd, nullptr, TRUE);
}

void RectEditor::OnMouseMove(HWND hWnd) {
    if (!isDragging) return;

    HDC hdc = GetDC(hWnd);
    SetROP2(hdc, R2_NOTXORPEN);
    HPEN hPen = CreateRubberPen();
    HBRUSH hBrush = static_cast<HBRUSH>(GetStockObject(NULL_BRUSH));
    HPEN hOldPen = static_cast<HPEN>(SelectObject(hdc, hPen));
    HBRUSH hOldBrush = static_cast<HBRUSH>(SelectObject(hdc, hBrush));

    Rectangle(hdc, 2 * x1 - x2, 2 * y1 - y2, x2, y2);

    POINT pt;
    GetCursorPos(&pt);
    ScreenToClient(hWnd, &pt);
    x2 = pt.x;
    y2 = pt.y;

    Rectangle(hdc, 2 * x1 - x2, 2 * y1 - y2, x2, y2);

    SelectObject(hdc, hOldBrush);
    SelectObject(hdc, hOldPen);
    DeleteObject(hPen);
    ReleaseDC(hWnd, hdc);
}

void RectEditor::OnLBup(HWND hWnd) {
    if (!isDragging) return;
    isDragging = false;
    ReleaseCapture();

    POINT pt;
    GetCursorPos(&pt);
    ScreenToClient(hWnd, &pt);
    x2 = pt.x;
```

```
    y2 = pt.y;

    if (shapeCount < MY_SHAPE_ARRAY_SIZE) {
        pcshape[shapeCount] = new RectShape();
        pcshape[shapeCount]->Set(x1, y1, x2, y2);
        ++shapeCount;
    }
    InvalidateRect(hWnd, nullptr, TRUE);
}

void EllipseEditor::OnMouseMove(HWND hWnd) {
    if (!isDragging) return;

    HDC hdc = GetDC(hWnd);
    SetROP2(hdc, R2_NOTXORPEN);
    HPEN hPen = CreateRubberPen();
    HBRUSH hBrush = static_cast<HBRUSH>(GetStockObject(NULL_BRUSH));
    HPEN hOldPen = static_cast<HPEN>(SelectObject(hdc, hPen));
    HBRUSH hOldBrush = static_cast<HBRUSH>(SelectObject(hdc, hBrush));

    Ellipse(hdc, x1, y1, x2, y2);

    POINT pt;
    GetCursorPos(&pt);
    ScreenToClient(hWnd, &pt);
    x2 = pt.x;
    y2 = pt.y;

    Ellipse(hdc, x1, y1, x2, y2);

    SelectObject(hdc, hOldBrush);
    SelectObject(hdc, hOldPen);
    DeleteObject(hPen);
    ReleaseDC(hWnd, hdc);
}

void EllipseEditor::OnLBup(HWND hWnd) {
    if (!isDragging) return;
    isDragging = false;
    ReleaseCapture();

    POINT pt;
    GetCursorPos(&pt);
    ScreenToClient(hWnd, &pt);
    x2 = pt.x;
    y2 = pt.y;

    if (shapeCount < MY_SHAPE_ARRAY_SIZE) {
        pcshape[shapeCount] = new EllipseShape();
        pcshape[shapeCount]->Set(x1, y1, x2, y2);
        ++shapeCount;
```

```
    }
    InvalidateRect(hWnd, nullptr, TRUE);
}

ShapeObjectsEditor::ShapeObjectsEditor() {
    ShapeEditor::InitArray();
    pse = nullptr;
}

ShapeObjectsEditor::~ShapeObjectsEditor() {
    delete pse;
    ShapeEditor::FreeArray();
}

void ShapeObjectsEditor::StartPointEditor(HWND hWnd) {
    delete pse;
    pse = new PointEditor();
    SetWindowTextW(hWnd, L"OOP_lab2 – Крапка");
}

void ShapeObjectsEditor::StartLineEditor(HWND hWnd) {
    delete pse;
    pse = new LineEditor();
    SetWindowTextW(hWnd, L"OOP_lab2 – Лінія");
}

void ShapeObjectsEditor::StartRectEditor(HWND hWnd) {
    delete pse;
    pse = new RectEditor();
    SetWindowTextW(hWnd, L"OOP_lab2 – Прямокутник");
}

void ShapeObjectsEditor::StartEllipseEditor(HWND hWnd) {
    delete pse;
    pse = new EllipseEditor();
    SetWindowTextW(hWnd, L"OOP_lab2 – Елiпс");
}

void ShapeObjectsEditor::OnLBdown(HWND hWnd) {
    if (pse) pse->OnLBdown(hWnd);
}

void ShapeObjectsEditor::OnLBup(HWND hWnd) {
    if (pse) pse->OnLBup(hWnd);
}

void ShapeObjectsEditor::OnMouseMove(HWND hWnd) {
    if (pse) pse->OnMouseMove(hWnd);
}

void ShapeObjectsEditor::OnPaint(HWND hWnd) {
```

```
    if (pse) {
        pse->OnPaint(hWnd);
    } else {
        PAINTSTRUCT ps;
        HDC hdc = BeginPaint(hWnd, &ps);
        ShapeEditor::DrawAllShapes(hdc);
        EndPaint(hWnd, &ps);
    }
}
```

shape_editor.h

```
#pragma once
#include <windows.h>
#include "editor.h"
#include "shape.h"

constexpr int MY_SHAPE_ARRAY_SIZE = 115;

class ShapeEditor : public Editor {
protected:
    long x1 = 0, y1 = 0, x2 = 0, y2 = 0;
    bool isDragging = false;

    static Shape** pcshape;
    static int shapeCount;

    HPEN CreateRubberPen() const;

public:
    static void InitArray();
    static void FreeArray();
    static void DrawAllShapes(HDC hdc);

    void OnLBdown(HWND hWnd) override;
    void OnPaint(HWND hWnd) override;
};

class PointEditor : public ShapeEditor {
public:
    void OnMouseMove(HWND hWnd) override;
    void OnLBup(HWND hWnd) override;
};

class LineEditor : public ShapeEditor {
public:
    void OnMouseMove(HWND hWnd) override;
    void OnLBup(HWND hWnd) override;
};
```

```
class RectEditor : public ShapeEditor {
public:
    void OnMouseMove(HWND hWnd) override;
    void OnLBup(HWND hWnd) override;
};

class EllipseEditor : public ShapeEditor {
public:
    void OnMouseMove(HWND hWnd) override;
    void OnLBup(HWND hWnd) override;
};

class ShapeObjectsEditor {
private:
    ShapeEditor* pse = nullptr;
public:
    ShapeObjectsEditor();
    ~ShapeObjectsEditor();

    void StartPointEditor(HWND hWnd);
    void StartLineEditor(HWND hWnd);
    void StartRectEditor(HWND hWnd);
    void StartEllipseEditor(HWND hWnd);

    void OnLBdown(HWND hWnd);
    void OnLBup(HWND hWnd);
    void OnMouseMove(HWND hWnd);
    void OnPaint(HWND hWnd);
};
```

editor.h

```
#pragma once
#include <windows.h>

class Editor {
public:
    virtual ~Editor() = default;
    virtual void OnLBdown(HWND hWnd) = 0;
    virtual void OnLBup(HWND hWnd) = 0;
    virtual void OnMouseMove(HWND hWnd) = 0;
    virtual void OnPaint(HWND hWnd) = 0;
};
```

resource.h

```
#pragma once
```

```
#define IDR_MAINMENU        101

#define ID_FILE_EXIT        40001

#define ID_OBJ_POINT        40002
#define ID_OBJ_LINE         40003
#define ID_OBJ_RECT         40004
#define ID_OBJ_ELLIPSE      40005

#define ID_HELP_ABOUT       40006
```

resource.rc

```
#pragma code_page(65001)
#include <windows.h>
#include "resource.h"

IDR_MAINMENU MENU
BEGIN
  POPUP "Файл"
  BEGIN
    MENUITEM "Вихід", ID_FILE_EXIT
  END
  POPUP "Об'єкти"
  BEGIN
    MENUITEM "Крапка",    ID_OBJ_POINT
    MENUITEM "Лінія",     ID_OBJ_LINE
    MENUITEM "Прямокутник", ID_OBJ_RECT
    MENUITEM "Еліпс",     ID_OBJ_ELLIPSE
  END
  POPUP "Довідка"
  BEGIN
    MENUITEM "Про програму...", ID_HELP_ABOUT
  END
END
```

Makefile

```
CXX       := g++
WINDRES   := windres
SRC_DIR   := src
RES_DIR   := res
BUILD_DIR := build

CXXFLAGS  := -std=c++17 -municode -DUNICODE -D_UNICODE -Wall -Wextra -I$(SRC_DIR)
LDFLAGS   := -municode -mwindows
LDLIBS    := -luser32 -lgdi32

TARGET    := Lab2.exe
SOURCES   := $(SRC_DIR)/Lab2.cpp $(SRC_DIR)/shape.cpp $(SRC_DIR)/shape_editor.cpp
OBJECTS   := $(patsubst $(SRC_DIR)/%.cpp,$(BUILD_DIR)/%.o,$(SOURCES))
RES_OBJ   := $(BUILD_DIR)/resource.res.o
HEADERS   := $(SRC_DIR)/resource.h $(SRC_DIR)/shape.h $(SRC_DIR)/editor.h
$(SRC_DIR)/shape_editor.h

.PHONY: all clean

all: $(TARGET)

$(TARGET): $(OBJECTS) $(RES_OBJ)
  $(CXX) $(OBJECTS) $(RES_OBJ) -o $@ $(LDFLAGS) $(LDLIBS)

$(BUILD_DIR)/%.o: $(SRC_DIR)/%.cpp $(HEADERS) | $(BUILD_DIR)
  $(CXX) $(CXXFLAGS) -c $< -o $@

$(RES_OBJ): $(RES_DIR)/resource.rc $(SRC_DIR)/resource.h | $(BUILD_DIR)
  $(WINDRES) -I$(SRC_DIR) $< -O coff -o $@

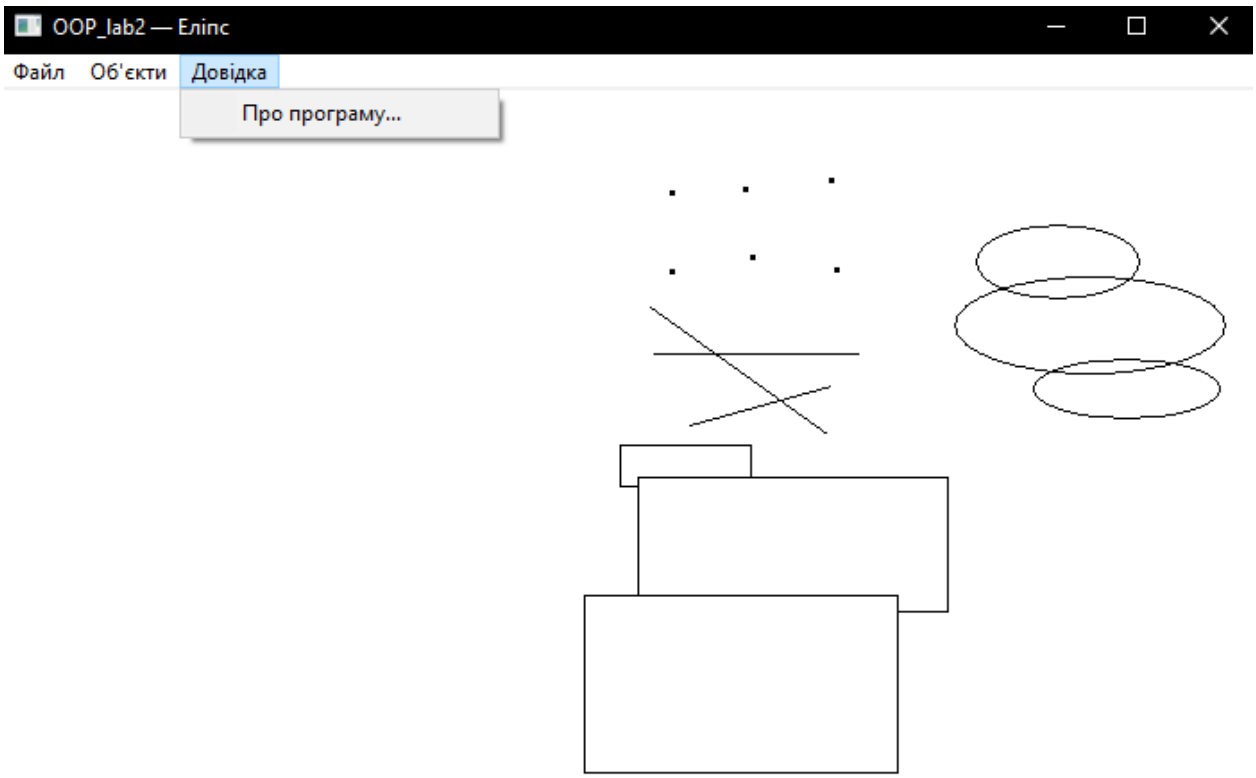
$(BUILD_DIR):
  mkdir -p $(BUILD_DIR)

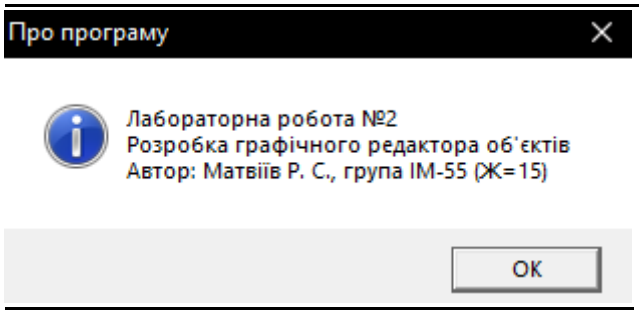
clean:
  rm -rf $(BUILD_DIR) $(TARGET)
```

Скріншоти


OOP_lab2 (Ж=15, ІМ-55)
Файл
Об'єкти
Довідка


OOP_lab2 (Ж=15, ІМ-55)
Файл
Об'єкти
Довідка
Крапка
Лінія
Прямокутник
Еліпс





Висновки

Під час виконання лабораторної роботи №2 я закріпив на практиці чотири основні принципи об’єктно-орієнтованого програмування: інкапсуляцію, абстракцію, успадкування та поліморфізм. Було розроблено графічний редактор на базі Win32 API та GDI, який підтримує малювання чотирьох типів геометричних примітивів із попереднім переглядом («гумовим» слідом) та збереженням об'єктів у динамічному поліморфному масиві.